这是一个整合了之前所有对话内容、补充了**逻辑层级验证**、**运单完整性验证**以及**用户权限安全性验证**的完整版指导文档。

你可以直接复制下面的内容，保存为 Markdown 文件（例如 Guide.md），或者直接转换成 PDF 发给你的队友。

智慧物流车队管理系统 - 部署与实验验证指南

适用对象：项目组成员
文档版本：v1.0 (Final)
最后更新：2025-12-29

第一部分：环境搭建 (Environment Setup)

在运行项目之前，请确保本地环境满足以下要求。

1. 基础软件

- **Python 3.8 或更高版本**：点击下载
- **SQL Server (2012-2022均可)**：确保本地服务已开启。
- **ODBC Driver 17 for SQL Server**：
  - 这是 Python 连接数据库的必装驱动。
  - 如果不确定是否安装，请点击下载安装。

2. 安装 Python 依赖库

打开 CMD 或 PowerShell，进入项目文件夹，运行：

Bash

```
pip install Flask pyodbc
```

第二部分：数据库部署 (Database Deployment)

请严格按照以下顺序初始化数据库，否则会导致功能缺失。

步骤 1：初始化基础库

1. 打开 SQL Server Management Studio (SSMS)。
2. 打开 Init_DB.sql (原始初始化脚本)。
3. **全选并执行 (F5)**。此时左侧应出现 LogisticsDB 数据库。

步骤 2：应用功能补丁 (Patch)

重要！ 必须执行此补丁以启用触发器、存储过程和状态流转逻辑。

1. 在 SSMS 中新建查询窗口。
2. 复制并执行以下 SQL 代码：

SQL

```
USE LogisticsDB;
GO

-- 1. 创建月度报表存储过程
IF OBJECT_ID('sp_GetFleetMonthlyReport', 'P') IS NOT NULL DROP PROCEDURE
sp_GetFleetMonthlyReport;
GO
CREATE PROCEDURE sp_GetFleetMonthlyReport
    @FleetID INT, @Year INT, @Month INT
AS
BEGIN
    SELECT f.FleetName, COUNT(DISTINCT o.OrderID) as TotalOrders,
ISNULL(SUM(a.FineAmount), 0) as TotalFines
    FROM Fleet f
    LEFT JOIN Vehicle v ON f.FleetID = v.FleetID
    LEFT JOIN Orders o ON v.VehicleID = o.VehicleID
    LEFT JOIN Abnormality a ON v.VehicleID = a.VehicleID AND MONTH(a.OccurTime)=@Month
    WHERE f.FleetID = @FleetID GROUP BY f.FleetName;
END;
GO

-- 2. 触发器：分配运单时车辆变"运输中"
IF OBJECT_ID('trg_SetVehicleBusy', 'TR') IS NOT NULL DROP TRIGGER trg_SetVehicleBusy;
```

```
GO
CREATE TRIGGER trg_SetVehicleBusy ON Orders AFTER INSERT AS
BEGIN
    UPDATE Vehicle SET CurrentStatus = '运输中' WHERE VehicleID IN (SELECT VehicleID FROM
inserted);
END;
GO

-- 3. 触发器：签收运单后车辆变"空闲"
IF OBJECT_ID('trg_SetVehicleIdle', 'TR') IS NOT NULL DROP TRIGGER trg_SetVehicleIdle;
GO
CREATE TRIGGER trg_SetVehicleIdle ON Orders AFTER UPDATE AS
BEGIN
    DECLARE @Vid INT; SELECT @Vid = VehicleID FROM inserted;
    IF NOT EXISTS (SELECT 1 FROM Orders WHERE VehicleID = @Vid AND Status <> '已完成')
        UPDATE Vehicle SET CurrentStatus = '空闲' WHERE VehicleID = @Vid;
END;
GO

-- 4. 触发器：司机信息修改审计
IF OBJECT_ID('trg_DriverAudit', 'TR') IS NOT NULL DROP TRIGGER trg_DriverAudit;
GO
CREATE TRIGGER trg_DriverAudit ON Driver AFTER UPDATE AS
BEGIN
    IF UPDATE(LicenseGrade)
        INSERT INTO History_Log (TableName, RecordID, OperationType, OldValue, NewValue)
        SELECT 'Driver', d.DriverID, 'Update_License', d.LicenseGrade, i.LicenseGrade
        FROM deleted d JOIN inserted i ON d.DriverID = i.DriverID;
END;
GO
```

步骤 3：配置权限验证环境 (Security Setup)

为了验证"司机无法修改数据"的安全性要求，需创建测试账号。请在 SSMS 中执行：

SQL

```
USE LogisticsDB;
GO
-- 创建"司机"登录名，密码为 StrongPass123!
IF NOT EXISTS (SELECT * FROM sys.server_principals WHERE name = 'Login_Driver')
```

```
    CREATE LOGIN Login_Driver WITH PASSWORD = 'StrongPass123!';

-- 创建数据库用户
IF NOT EXISTS (SELECT * FROM sys.database_principals WHERE name = 'User_Driver')
    CREATE USER User_Driver FOR LOGIN Login_Driver;

-- 关键：拒绝司机修改数据的权限
DENY UPDATE, INSERT, DELETE ON SCHEMA::dbo TO User_Driver;
GO
```

第三部分：代码框架与运行 (Run Application)

请确保你的文件目录结构如下所示，**文件名必须完全一致**：

Plaintext

```
LogisticsProject/
│
├── app.py              # 后端启动入口 (已更新包含 signoff 路由)
├── database.py         # 数据库操作类 (已更新包含 complete_order 方法)
│
└── templates/          # 前端页面
    ├── layout.html     # 布局 (已更新侧边栏包含"运单签收")
    ├── index.html      # 首页
    ├── allocate.html   # 运单分配页
    ├── entry.html      # 基础录入页 (已包含正则校验)
    ├── exception.html  # 异常管理页
    ├── report.html     # 报表页
    └── signoff.html    # 签收页 (新增文件)
```

启动方式：

1. 在项目根目录打开终端。
2. 运行 python app.py。
3. 浏览器访问 http://127.0.0.1:5000。

第四部分：功能验证清单 (Verification Checklist)

请对照此清单进行演示，覆盖所有课程设计要求。

✅ 1. 基础信息校验 (对应要求 II.1)

- [ ] **操作**：进入“基础录入” -> “录入车辆”。
- [ ] **测试点 A**：输入车牌 12345 并提交。
  - **预期**：浏览器弹窗提示“请与所请求的格式保持一致” (正则校验生效)。
- [ ] **测试点 B**：输入正确车牌 京A88888 并提交。
  - **预期**：提示录入成功。
- [ ] **测试点 C**：进入“录入司机”，尝试手动输入驾照等级。
  - **预期**：无法输入，只能从下拉框选择 A1/A2 等 (枚举校验生效)。

✅ 2. 运单分配与超载拦截 (对应要求 II.2 & III.3)

- [ ] **操作**：进入“运单分配”。
- [ ] **准备**：选择一辆限重 5000kg 的空闲车辆。
- [ ] **测试点 A (正常分配)**：分配 100kg 货物。
  - **预期**：提示成功。
  - **验证**：去 SSMS 查询 SELECT CurrentStatus FROM Vehicle，该车状态应变为 **'运输中'** (Trigger自动流转生效)。
- [ ] **测试点 B (超载拦截)**：给同一辆车再分配 6000kg 货物。
  - **预期**：页面顶部出现**红色警告条**：“【操作拒绝】严重超载...”。
  - **原理**：Python 捕获了 SQL Trigger 抛出的 RAISERROR。

✅ 3. 运单签收与状态闭环 (对应要求 III.3)

- [ ] **操作**：点击左侧导航栏 **“运单签收”**。
- [ ] **测试点**：找到刚才分配的运单，点击“确认签收”。
- [ ] **预期**：提示签收成功。
- [ ] **验证**：去 SSMS 查询该车状态，应自动变回 **'空闲'** (Trigger自动流转生效)。

✅ 4. 异常录入 (对应要求 II.3)

- [ ] **操作**：进入“异常管理”。
- [ ] **测试点**：下拉框能正常显示车辆和司机，录入并提交成功。

✅ 5. 审计日志 (对应要求 III.3)

- [ ] **操作**：在 SSMS 中执行 SQL 修改语句：

```sql
UPDATE Driver SET LicenseGrade = 'C1' WHERE DriverID = 1;
```

- [ ] **验证**：查询 SELECT * FROM History_Log，应出现一条新记录，记录了旧驾照等级。

✅ 6. 统计报表与存储过程 (对应要求 II.6 & III.3)

- [ ] **操作 A (前端)**：进入“数据报表”，查看每周异常视图数据。
- [ ] **操作 B (后端)**：在 SSMS 中执行存储过程：
  SQL
  ```
  EXEC sp_GetFleetMonthlyReport @FleetID=1, @Year=2025, @Month=12;
  ```
  - **预期**：返回包含“总运单数”、“罚款总额”的统计表格。

✅ 7. 索引优化 (对应要求 III.4)

- [ ] **操作**：在 SSMS 中演示 SELECT * FROM Vehicle WHERE PlateNumber = '...'。
- [ ] **验证**：开启“执行计划”，对比建立索引前后的查询开销（Table Scan vs Index Seek）。

✅ 8. 配送中心与车队层级关系 (对应语义规则 1 & 2)

- [ ] **操作**：在 SSMS 中执行以下 SQL，验证 1:N 关系及主管身份。
  SQL
  ```
  SELECT DC.Name AS [配送中心], F.FleetName AS [下辖车队], F.SupervisorName AS [调度主管]
  FROM DistributionCenter DC
  JOIN Fleet F ON DC.CenterID = F.CenterID;
  ```
- [ ] **预期**：清晰展示中心与车队的层级，证明逻辑结构设计合理。

✅ 9. 运单详情完整性验证 (对应语义规则 3)

- [ ] **操作**：在 Web 端分配一个目的地为“测试上海”的运单。
- [ ] **验证**：在 SSMS 中执行以下 SQL：
  SQL
  ```
  SELECT TOP 1 OrderID, Weight, Destination FROM Orders ORDER BY OrderID DESC;
  ```
- [ ] **预期**：数据库中 Destination 字段准确显示为“测试上海”，证明前后端数据流转无误。

✅ 10. 安全性权限验证 (对应语义规则 4)

- [ ] **说明**：验证“司机”账号只能查询，无法修改数据。
- [ ] **操作 1 (登录)**：在 SSMS 点击“连接” -> “数据库引擎”，使用 **SQL Server 身份验证**。
  - 登录名：Login_Driver
  - 密码：StrongPass123!
- [ ] **操作 2 (尝试修改)**：在新打开的窗口执行：
  SQL
  ```
  USE LogisticsDB;
  UPDATE Vehicle SET MaxWeight = 99999 WHERE VehicleID = 1;
  ```

- [ ] **预期结果：报错** 拒绝了对对象 'Vehicle' ... 的 UPDATE 权限。
  - *(这是数据库安全性设计的核心得分点)*

⚠️ 常见问题排查

1. **报错 Data source name not found**:
   - **原因**：没装 ODBC Driver 17。
   - **解决**：下载安装驱动，或在 database.py 中将 DRIVER={...} 改为 DRIVER={SQL Server}。
2. **报错 Login failed**:
   - **原因**：数据库连接不通。
   - **解决**：检查 database.py 中的 SERVER 是否为 localhost，如果不行为改为计算机名。
3. **权限验证时不报错**:
   - **原因**：你可能还在用默认的管理员窗口执行代码。
   - **解决**：务必使用 Login_Driver 账号建立**新连接**后再执行 UPDATE 语句。

文档结束